INTERVIEW PREP SERIES

Python

Interview Questions & Easy Answers

---

55 questions · 25 marked ★ most asked

Covers Python basics, data types, mutability, lists vs tuples, dictionaries and sets, functions, *args/**kwargs, lambda, comprehensions, OOP, decorators, generators, exceptions, memory management, the GIL, modules and the output-based questions asked in service-company and startup interviews.

How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

Basics

Q1. What is Python? What are its key features? ★ Most asked

Python is a high-level, interpreted, general-purpose language known for simple, readable syntax. Features: dynamically typed, object-oriented, huge standard library, cross-platform, automatic memory management, and huge community libraries for web (Django, Flask, FastAPI), data science (Pandas, NumPy) and AI (PyTorch).

Q2. Is Python compiled or interpreted? ★ Most asked

Both, in a way. Python source code is first compiled into **bytecode** (`.pyc` files), and then the **Python Virtual Machine** interprets that bytecode line by line. Because there's no separate compile step for the user, it's called an interpreted language.

Q3. What are Python's built-in data types? ★ Most asked

- Numbers: `int`, `float`, `complex`
- Text: `str`
- Sequences: `list`, `tuple`, `range`
- Mapping: `dict`
- Sets: `set`, `frozenset`
- Boolean: `bool`
- `NoneType` (`None`)
- Binary: `bytes`, `bytearray`

Q4. What is the difference between mutable and immutable types? ★ Most asked

Mutable objects can be changed after creation: `list`, `dict`, `set`. Immutable objects can't: `int`, `float`, `str`, `tuple`, `frozenset`. "Changing" an immutable object actually creates a new object.

```
s = "hi"; s += "!"   # creates a new string
a = [1]; a.append(2) # same list changed
```

Q5. What is dynamic typing?

You don't declare variable types; the type is decided at runtime from the value, and a variable can hold different types over time: `x = 5` then `x = "five"`. Type hints (`x: int = 5`) are optional and only help readability and tools.

Q6. What is PEP 8?

Python's official style guide: 4-space indentation, `snake_case` for variables and functions, `PascalCase` for classes, lines under 79-99 characters, and spaces around operators.

Q7. Why is indentation important in Python?

Python uses indentation instead of `{ }` to define code blocks (if, for, functions, classes). Wrong indentation gives an `IndentationError` or changes the program's logic.

Lists, tuples, dicts and sets

Q8. What is the difference between a list and a tuple? ★ Most asked

- List: mutable, written with `[ ]`, slightly slower, more memory, has many methods (append, remove, sort).

- Tuple: immutable, written with `( )`, faster, less memory, can be used as a dictionary key (if its items are immutable).

Use tuples for fixed data like coordinates `(x, y)` and lists for collections that change.

Q9. What is the difference between a list and an array? ★ Most asked

A Python list can hold mixed types and is flexible. An array (from the `array` module or NumPy) holds a single type, uses less memory and is much faster for math. NumPy arrays support vectorised operations like `arr * 2`.

Interview tip: "What is list, difference between list and array" was a reported Revature interview question on Glassdoor.

Q10. What is a dictionary? How do you loop through it? ★ Most asked

A dictionary stores key-value pairs with O(1) average lookup. Keys must be immutable (hashable). Since Python 3.7, dicts keep insertion order.

```
student = {"name": "Asha", "age": 22}
student["city"] = "Delhi"
for key, value in student.items():
    print(key, value)
student.get("phone", "N/A")   # safe access with default
```

Q11. What is a set and when is it useful?

A set is an unordered collection of unique items. It's useful to remove duplicates and for fast membership checks (O(1)). It supports union `|`, intersection `&` and difference `-`.

```
unique = set([1, 2, 2, 3])   # {1, 2, 3}
```

Q12. What is slicing? ★ Most asked

Getting a part of a sequence with `[start:stop:step]`. The stop index is not included.

```
s = "python"
s[0:3]    # 'pyt'
s[-3:]    # 'hon'
s[::-1]   # 'nohtyp' (reverse)
```

Q13. What is the difference between `append()`, `extend()` and `insert()`?

`append(x)` adds x as one item at the end. `extend(iterable)` adds each item of the iterable. `insert(i, x)` adds x at index i.

```
a = [1, 2]; a.append([3, 4])  # [1, 2, [3, 4]]
b = [1, 2]; b.extend([3, 4])  # [1, 2, 3, 4]
```

Q14. What is the difference between `remove()`, `pop()` and `del`?

`remove(x)` deletes the first item equal to x. `pop(i)` removes the item at index i (default last) and returns it. `del a[i]` deletes by index or slice and returns nothing.

Q15. What is the difference between `sort()` and `sorted()`?

`list.sort()` sorts the list in place and returns `None`. `sorted(iterable)` returns a new sorted list and works on any iterable. Both accept `key=` and `reverse=True`.

Functions

Q16. What are *args and **kwargs? ★ Most asked

*args collects extra positional arguments into a tuple. **kwargs collects extra keyword arguments into a dictionary.

```
def info(*args, **kwargs):
    print(args)    # (1, 2)
    print(kwargs)  # {'name': 'Asha'}

info(1, 2, name="Asha")
```

Q17. What is a lambda function? ★ Most asked

A small anonymous function written in one line, with one expression.

```
square = lambda x: x * x
nums = sorted(users, key=lambda u: u["age"])
```

Q18. What are map(), filter() and reduce()? ★ Most asked

- `map(func, iterable)` applies func to every item.
- `filter(func, iterable)` keeps items where func returns True.
- `reduce(func, iterable)` (from `functools`) combines items into one value.

```
list(map(lambda x: x * 2, [1, 2, 3]))         # [2, 4, 6]
list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4]))  # [2, 4]
from functools import reduce
reduce(lambda a, b: a + b, [1, 2, 3])          # 6
```

Q19. What is a list comprehension? ★ Most asked

A short way to create lists from another iterable, often with a condition.

```
squares = [x * x for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
```

There are also dict comprehensions `{k: v for ...}` and set comprehensions `{x for ...}`.

Q20. What is the output of this code? (mutable default argument) ★ Most asked

```
def add(item, lst=[]):
    lst.append(item)
    return lst

print(add(1))
print(add(2))
```

Output: `[1]` then `[1, 2]`. The default list is created **once** when the function is defined and reused on every call. Fix it with `lst=None` and inside: `if lst is None: lst = []`.

Q21. How are arguments passed in Python: by value or by reference?

Python uses "pass by object reference". The function gets a reference to the same object. If the object is mutable (list), changes inside the function affect the original. If it's immutable (int, str), reassigning inside the function creates a new local object.

Q22. What is the difference between `global` and `nonlocal`?

`global x` lets a function change a variable defined at the module level. `nonlocal x` lets a nested function change a variable from its enclosing function.

Q23. What is the LEGB rule?

The order Python searches for a variable name: **L**ocal → **E**nclosing (outer function) → **G**lobal → **B**uilt-in.

Q24. What is recursion? What is the recursion limit?

A function calling itself with a base case to stop. Python's default recursion limit is about 1000 calls (`sys.getrecursionlimit()`); going beyond raises `RecursionError`. Python doesn't optimise tail recursion, so deep recursion is better rewritten as a loop.

OOP

Q25. What are classes and objects? What is `self`? ★ Most asked

A class is a blueprint; an object is an instance created from it. `self` refers to the current object and is used to access its attributes and methods. `__init__` is the constructor that runs when an object is created.

```
class Car:
    def __init__(self, brand):
        self.brand = brand
    def drive(self):
        return f"{self.brand} is driving"

c = Car("Tata")
c.drive()
```

Q26. What are the four pillars of OOP in Python? ★ Most asked

- **Encapsulation**: bundling data and methods in a class; hiding details with `_protected` and `__private` names.
- **Abstraction**: showing only essential features; using abstract base classes (`abc` module).
- **Inheritance**: a child class reuses a parent class: `class Dog(Animal):`.
- **Polymorphism**: the same method name behaves differently in different classes (method overriding, duck typing).

Q27. What types of inheritance does Python support?

Single, multiple (`class C(A, B)`), multilevel (A → B → C), hierarchical (many children from one parent) and hybrid. For multiple inheritance, Python decides which method to use with the **MRO** (Method Resolution Order), visible via `ClassName.__mro__`.

Q28. What does `super()` do?

`super()` calls a method from the parent class, most often the parent's `__init__`:

```
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
```

Q29. What is the difference between instance, class and static methods? ★ Most asked

- **Instance method**: takes `self`; works with object data.
- **Class method** (`@classmethod`): takes `cls`; works with class-level data; often used as alternative constructors.
- **Static method** (`@staticmethod`): takes neither; a utility function that just lives inside the class.

Q30. What are dunder (magic) methods?

Special methods with double underscores that let your classes work with Python's built-in operations: `__init__` (constructor), `__str__` (print-friendly text), `__repr__` (developer text), `__len__`, `__eq__`, `__add__` (operator overloading), `__getitem__`.

Q31. Does Python support method overloading?

Not in the traditional way: if you define two methods with the same name, the last one replaces the first. You get the same effect with default arguments or `*args`. Method **overriding** (child class redefining a parent's method) is fully supported.

Q32. How do you make an attribute private?

Prefix it with double underscores (`self.__salary`). Python then "name-mangles" it to `_ClassName__salary`, so it's hard to access from outside. A single underscore (`_salary`) is only a convention meaning "internal use".

Advanced features

Q33. What is a decorator? ★ Most asked

A decorator is a function that takes another function and adds extra behaviour to it, without changing its code. It's applied with `@`.

```
def logger(func):
    def wrapper(*args, **kwargs):
        print("Calling", func.__name__)
        return func(*args, **kwargs)
    return wrapper

@logger
def greet(name):
    return "Hi " + name
```

Common real uses: `@login_required` in Django, `@app.route` in Flask, timing and caching (`@lru_cache`).

Q34. What is a generator? What does `yield` do? ★ Most asked

A generator is a function that returns values one at a time using `yield`, pausing between them. It doesn't store everything in memory, so it's great for large data or infinite sequences.

```
def count_up(n):
    for i in range(n):
        yield i

for x in count_up(3):
    print(x)   # 0 1 2
```

Q35. What is the difference between an iterator and an iterable?

An **iterable** is anything you can loop over (list, string, dict); it has `__iter__()`. An **iterator** is the object that actually produces the next value with `__next__()` and remembers its position. `iter(my_list)` gives an iterator; generators are iterators.

Q36. What is the difference between `return` and `yield`?

`return` sends back a value and ends the function. `yield` sends back a value and **pauses** the function, so it can continue from there on the next call.

Q37. What is the difference between `is` and `==`? ★ Most asked

`==` checks if two values are equal. `is` checks if both variables point to the **same object** in memory.

```
a = [1, 2]; b = [1, 2]
a == b   # True
a is b   # False
```

Use `is` for comparing with `None`: `if x is None:`.

Q38. What is the difference between shallow copy and deep copy? ★ Most asked

A shallow copy creates a new outer object but shares the inner (nested) objects. A deep copy copies everything recursively.

```
import copy
a = [[1, 2], [3, 4]]
s = copy.copy(a);  s[0].append(9)   # a changes too
d = copy.deepcopy(a); d[0].append(9)  # a unchanged
```

Q39. What are context managers and the `with` statement?

Context managers automatically set up and clean up resources. `with open("f.txt") as f:` closes the file automatically, even if an error happens. You can build your own with `__enter__`/`__exit__` or `@contextmanager`.

Q40. What is `if __name__ == "__main__":`?

Code inside this block runs only when the file is executed directly, not when it's imported as a module. It's used to keep test or startup code from running on import.

Errors, memory and concurrency

Q41. How do you handle exceptions in Python? ★ Most asked

```
try:
    num = int(input())
    result = 10 / num
except ValueError:
    print("Not a number")
except ZeroDivisionError:
    print("Can't divide by zero")
else:
    print("Result:", result)   # runs if no error
finally:
    print("Always runs")       # cleanup
```

Raise your own with `raise ValueError("message")`, and create custom exceptions by inheriting from `Exception`.

Q42. How is memory managed in Python? ★ Most asked

Python manages memory automatically using a private heap. It mainly uses **reference counting**: each object tracks how many references point to it, and it's freed when the count hits 0. A **garbage collector** (the `gc` module) also cleans up reference cycles (objects pointing to each other).

Q43. What is the GIL (Global Interpreter Lock)? ★ Most asked

The GIL is a lock in CPython that lets only one thread run Python bytecode at a time. So threads don't speed up CPU-heavy work, but they still help for I/O-bound work (network, files) because the GIL is released while waiting. For CPU-heavy tasks, use `multiprocessing`. (Python 3.13+ offers an optional free-threaded build without the GIL.)

Q44. What is the difference between multithreading and multiprocessing in Python?

Threads share memory and are light but are limited by the GIL, so they're best for I/O-bound tasks. Processes each have their own memory and Python interpreter, so they run truly in parallel and are best for CPU-bound tasks, at the cost of more memory.

Q45. What is `async`/`await` in Python?

`asyncio` lets one thread handle many I/O tasks at the same time by switching between them while they wait. Functions defined with `async def` are coroutines, and `await` pauses one until the result is ready. Used in FastAPI and web scraping.

Modules and practical

Q46. What is the difference between a module and a package?

A module is a single `.py` file. A package is a folder of modules (traditionally with an `__init__.py` file). You install third-party packages with `pip install name`.

Q47. What is a virtual environment and why use it?

An isolated Python environment for each project, with its own packages and versions, so projects don't conflict. Create with `python -m venv venv`, activate it, and save dependencies with `pip freeze > requirements.txt`.

Q48. Reverse a string and check for a palindrome in Python. ★ Most asked

```
s = "madam"
print(s[::-1])            # reverse
print(s == s[::-1])       # True -> palindrome
```

Q49. Count the frequency of each character or word. ★ Most asked

```
from collections import Counter
Counter("banana")                # {'a': 3, 'n': 2, 'b': 1}
Counter("a b a".split()).most_common(1)  # [('a', 2)]
```

Q50. What are some useful modules from `collections`?

`Counter` (counting), `defaultdict` (dict with a default value for missing keys), `deque` (fast append/pop from both ends), `namedtuple` (tuples with named fields), `OrderedDict`.

Q51. How do you read and write files?

```
with open("data.txt", "w") as f:
    f.write("Hello\n")
with open("data.txt") as f:
    for line in f:
        print(line.strip())
```

Modes: `r` read, `w` write (overwrite), `a` append, `rb`/`wb` binary.

Q52. What are `pass`, `break` and `continue`?

`pass` does nothing (a placeholder). `break` exits the loop immediately. `continue` skips the rest of the current iteration and moves to the next one.

Q53. What is the difference between Python 2 and Python 3?

Python 2 reached end of life in 2020. Python 3 has `print()` as a function, true division (`5/2 = 2.5`), Unicode strings by default, and modern features like f-strings and type hints. Always use Python 3.

Q54. What are f-strings?

A fast, readable way to put values in strings (Python 3.6+): `f"Name: {name}, Age: {age + 1}"`. You can format numbers too: `f"{price:.2f}"`.

Q55. What is the difference between Django, Flask and FastAPI?

Django is a "batteries-included" full framework with ORM, admin panel and auth. Flask is a lightweight micro-framework where you choose your own tools. FastAPI is a modern, fast framework for building APIs with automatic docs, async support and type-hint validation.